

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Informe técnico individual

Práctica de Bases de Datos Distribuidas

Caso: Banco del Austro

Estudiante:	José Alejandro Lozano Morales
Asignatura:	Aplicaciones Distribuidas (ISR-701)
Docente:	Ph.D. Gleiston Guerrero Ulloa
Período académico:	2026-2027 SPA
Fecha:	14 de julio de 2026

Índice

1. Resumen	3
2. Introducción	3
3. Objetivos	3
3.1. Objetivo general	3
3.2. Objetivos específicos	3
4. Fundamentos de la distribución	4
4.1. Nodos y sedes	4
4.2. Estrategia de fragmentación	4
5. Arquitectura de la solución	4
6. Implementación	5
6.1. Despliegue de PostgreSQL	5
6.2. Configuración de Spring Boot	5
6.3. Justificación de diferencias respecto a la guía	5
6.3.1. Propiedad del controlador en <code>application.yml</code>	6
6.3.2. Refactorización de <code>DataSourceConfig</code>	6
6.3.3. Refactorización de <code>ConsultaDistribuidaService</code>	6
6.4. API REST	6
7. Pruebas y resultados	7
7.1. Estado inicial de los nodos	7
7.2. Consulta del nodo Cuenca	7
7.3. Consulta del nodo Quito	8
7.4. Consulta distribuida de clientes	9
7.5. Prueba de autonomía local	10
8. Análisis de resultados	12
9. Conclusiones	12

10.Recomendaciones	12
11.Referencias	13

1. Resumen

En esta práctica se implementó un prototipo de base de datos distribuida para dos oficinas del Banco del Austro: Cuenca y Quito. Cada sede utiliza una instancia independiente de PostgreSQL 16 desplegada mediante Docker. Una aplicación desarrollada con Spring Boot consulta ambos nodos y selecciona la ubicación de una cuenta a partir de su prefijo. Las cuentas que empiezan con 22 pertenecen a Cuenca y las que empiezan con 17 pertenecen a Quito.

La solución permitió comprobar la fragmentación horizontal, la transparencia de ubicación y la autonomía local. Las pruebas funcionales confirmaron la consulta correcta de saldos en ambos nodos, la recuperación conjunta de seis clientes y la continuidad de las consultas de Cuenca durante una interrupción controlada del nodo Quito.

Palabras clave: base de datos distribuida, fragmentación horizontal, PostgreSQL, Docker, Spring Boot, autonomía local.

2. Introducción

Una base de datos distribuida almacena información en varios nodos físicos o lógicos, pero busca ofrecer al usuario una vista unificada del sistema. Esta organización permite acercar los datos a las sedes que los utilizan, distribuir la carga y mantener operaciones locales cuando otro nodo no está disponible.

El caso propuesto representa dos oficinas del Banco del Austro. La información de cuentas y transacciones se separa por oficina, mientras que la aplicación cliente oculta la ubicación física de cada registro. El consumidor de la API utiliza una sola dirección y no necesita conocer qué instancia PostgreSQL atiende la petición.

3. Objetivos

3.1. Objetivo general

Implementar un prototipo de base de datos distribuida para consultar información bancaria almacenada en dos nodos PostgreSQL independientes.

3.2. Objetivos específicos

- Desplegar los nodos de Cuenca y Quito mediante Docker Compose.
- Aplicar fragmentación horizontal a cuentas y transacciones según su oficina.
- Configurar dos fuentes de datos en una aplicación Spring Boot.
- Enrutar las consultas mediante los prefijos 22 y 17.

- Exponer endpoints REST para consultar saldos y clientes.
- Verificar la autonomía local mediante la interrupción temporal de un nodo.

4. Fundamentos de la distribución

4.1. Nodos y sedes

Cada sede corresponde a un nodo PostgreSQL autónomo. El nodo Cuenca administra la base de datos `banco_cuenca`, mientras que el nodo Quito administra `banco_quito`. Los datos se almacenan en volúmenes separados para conservarlos aunque los contenedores sean detenidos o recreados.

4.2. Estrategia de fragmentación

La tabla `cuentas` se fragmenta horizontalmente por oficina. El nodo Cuenca acepta únicamente filas cuya oficina sea `CUENCA`; el nodo Quito aplica la misma regla con `QUITO`. Las restricciones `CHECK` protegen esta condición directamente en el motor de base de datos.

Cuadro 1: Distribución de información por nodo

Nodo	Prefijo de cuenta	Oficina
Cuenca	22	CUENCA
Quito	17	QUITO

5. Arquitectura de la solución

La solución está formada por dos contenedores PostgreSQL y una aplicación Spring Boot. Aunque ambos motores escuchan en el puerto interno 5432, Docker los publica en puertos diferentes de la computadora anfitriona.

Cuadro 2: Componentes y puertos utilizados

Componente	Dirección	Responsabilidad
PostgreSQL Cuenca	<code>localhost:5442</code>	Cuentas con prefijo 22
PostgreSQL Quito	<code>localhost:5443</code>	Cuentas con prefijo 17
Aplicación Spring Boot	<code>localhost:8080</code>	API REST y enrutamiento

Los puertos externos 5442 y 5443 siguen la alternativa indicada en la guía para equipos

donde el puerto 5432 ya está ocupado por una instalación local de PostgreSQL. Los puertos internos de los contenedores no fueron modificados.

6. Implementación

6.1. Despliegue de PostgreSQL

El archivo `docker-compose.yml` declara los servicios `db-cuenca` y `db-quito`, sus credenciales académicas, volúmenes persistentes, scripts de inicialización y una red común. Los archivos `01_schema.sql` crean la estructura y los archivos `02_datos.sql` insertan los registros de prueba.

6.2. Configuración de Spring Boot

La clase `DataSourceConfig` construye dos objetos `DataSource`. La configuración de conexión se encuentra en `application.yml`. El servicio `ConsultaDistribuidaService` crea un `JdbcTemplate` por sede y selecciona uno mediante los dos primeros caracteres del número de cuenta.

El proyecto fue configurado en IntelliJ IDEA con Microsoft OpenJDK 21.0.11 y nivel de lenguaje 21, de acuerdo con los requisitos establecidos para la práctica.

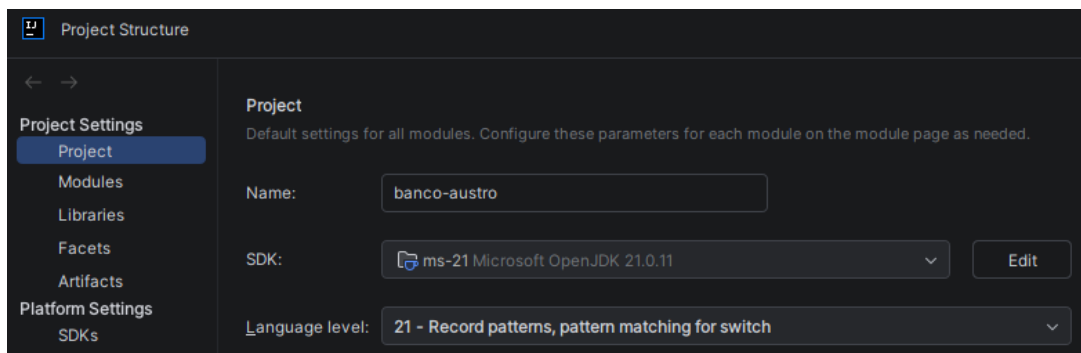


Figura 1: Configuración del proyecto con JDK 21 y nivel de lenguaje 21 en IntelliJ IDEA.

6.3. Justificación de diferencias respecto a la guía

La implementación conserva el comportamiento y los resultados solicitados en la guía, pero introduce pequeñas refactorizaciones para evitar configuración redundante y aprovechar las capacidades de Java 21. A continuación se justifican las diferencias principales.

6.3.1. Propiedad del controlador en `application.yml`

El listado de la guía incluye la propiedad `driver-class-name: org.postgresql.Driver` para cada fuente de datos. En este proyecto dicha línea se omitió porque `DataSourceConfig` establece explícitamente el mismo controlador mediante `driverClassName(.'org.postgresql.Driver)` al construir ambos objetos `DataSource`. La clase del controlador está disponible gracias a la dependencia de PostgreSQL declarada en Maven.

La configuración utiliza el espacio de propiedades personalizado `datasources` y no la configuración automática singular `spring.datasource`. Además, `DataSourceConfig` inyecta únicamente URL, usuario y contraseña. Por ello, añadir `driver-class-name` al YAML sin inyectarlo también en la clase dejaría una propiedad no consumida. Su omisión evita duplicar una configuración que ya se aplica de forma centralizada y no altera las conexiones comprobadas con Cuenca y Quito.

6.3.2. Refactorización de `DataSourceConfig`

La guía inyecta URL, usuario y contraseña en seis campos de la clase y repite la construcción de cada fuente de datos. La versión implementada recibe esas propiedades directamente como parámetros de los métodos `dsCuenca` y `dsQuito`. De esta manera, las dependencias de cada bean quedan visibles en su declaración y no se requieren campos mutables de configuración.

La lógica compartida se trasladó al método privado `construir`, que configura URL, credenciales y controlador en un único lugar. Esta decisión reduce duplicación: cualquier modificación común a las dos conexiones se realiza una sola vez. Los nombres de los beans, `dsCuenca` y `dsQuito`, permanecen idénticos a los de la guía, por lo que el uso de `@Qualifier` y el comportamiento de Spring no cambian.

6.3.3. Refactorización de `ConsultaDistribuidaService`

El servicio conserva las mismas consultas SQL, la unión de clientes y las reglas de enrutamiento: prefijo 22 para Cuenca y prefijo 17 para Quito. La diferencia se limita a expresar la respuesta de `consultarSaldo` mediante un operador condicional en lugar de un bloque `if`. Cuando la consulta no devuelve filas se construye el mismo mapa de error; cuando existe una fila se retorna el primer resultado.

La guía obtiene esa fila con `get(0)`, mientras que la implementación usa `getFirst()`. Este método forma parte de la API disponible en Java 21, versión exigida por la práctica, y expresa directamente la intención de recuperar el primer elemento. También se escribió la consulta SQL como una sola cadena en vez de concatenar dos fragmentos. Estas diferencias mejoran concisión y legibilidad sin modificar el resultado funcional.

6.4. API REST

La clase `BancoController` publica los siguientes endpoints:

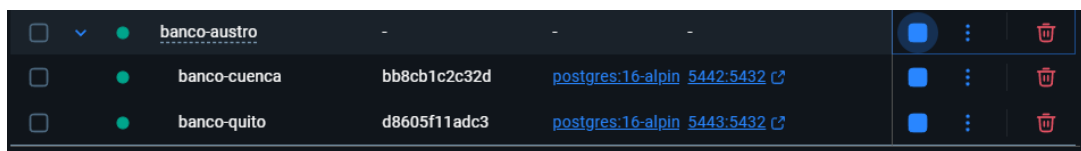
Cuadro 3: Endpoints implementados

Endpoint	Resultado
GET /api/banco/saldo/2201000001	Saldo almacenado en Cuenca
GET /api/banco/saldo/1701000001	Saldo almacenado en Quito
GET /api/banco/clientes	Unión de clientes de ambos nodos

7. Pruebas y resultados

7.1. Estado inicial de los nodos

Docker Compose levantó correctamente ambos contenedores. Cuenca quedó publicado en el puerto 5442 y Quito en el puerto 5443.



☐	▼	●	banco-austro	-	-	-			
☐		●	banco-cuenca	bb8cb1c2c32d	postgres:16-alpin	5442:5432			
☐		●	banco-quito	d8605f11adc3	postgres:16-alpin	5443:5432			

Figura 2: Contenedores de Cuenca y Quito en ejecución.

7.2. Consulta del nodo Cuenca

La petición de la cuenta 2201000001 devolvió HTTP 200, un saldo de 15000.00 y la oficina CUENCA.

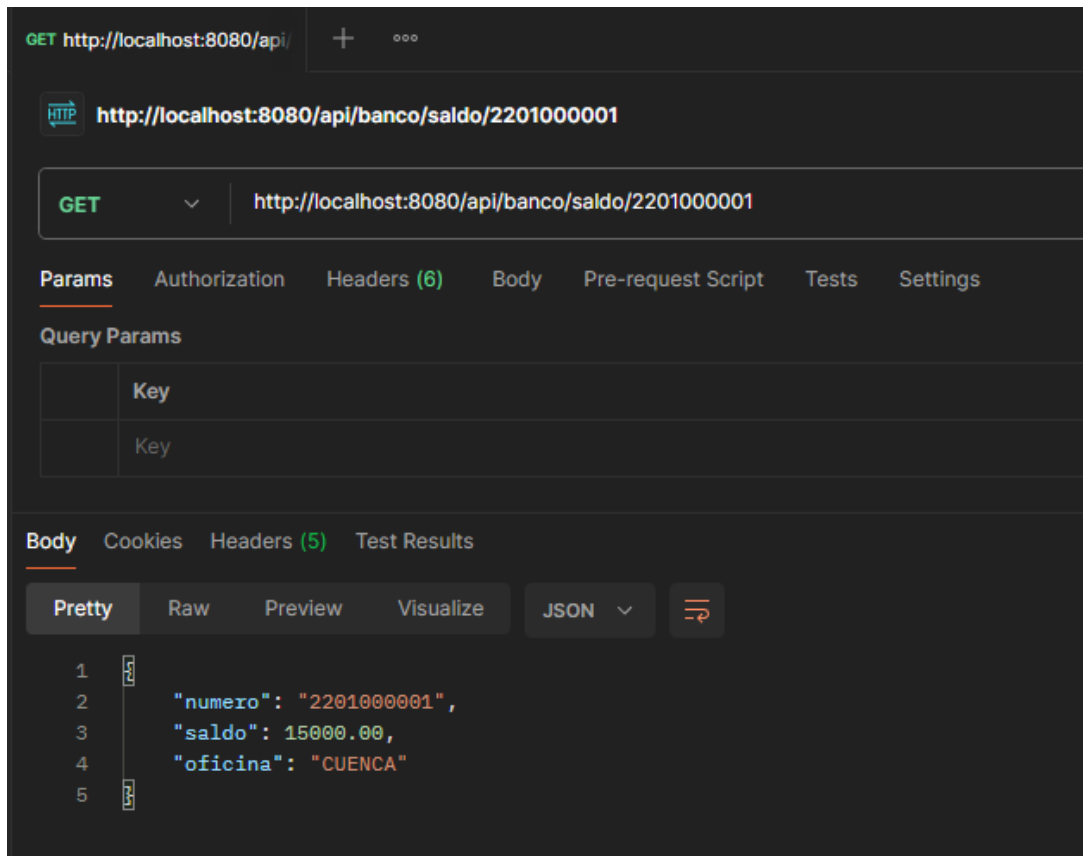


Figura 3: Consulta de saldo dirigida al nodo Cuenca.

7.3. Consulta del nodo Quito

La petición de la cuenta 1701000001 devolvió HTTP 200, un saldo de 8000.00 y la oficina QUITO.

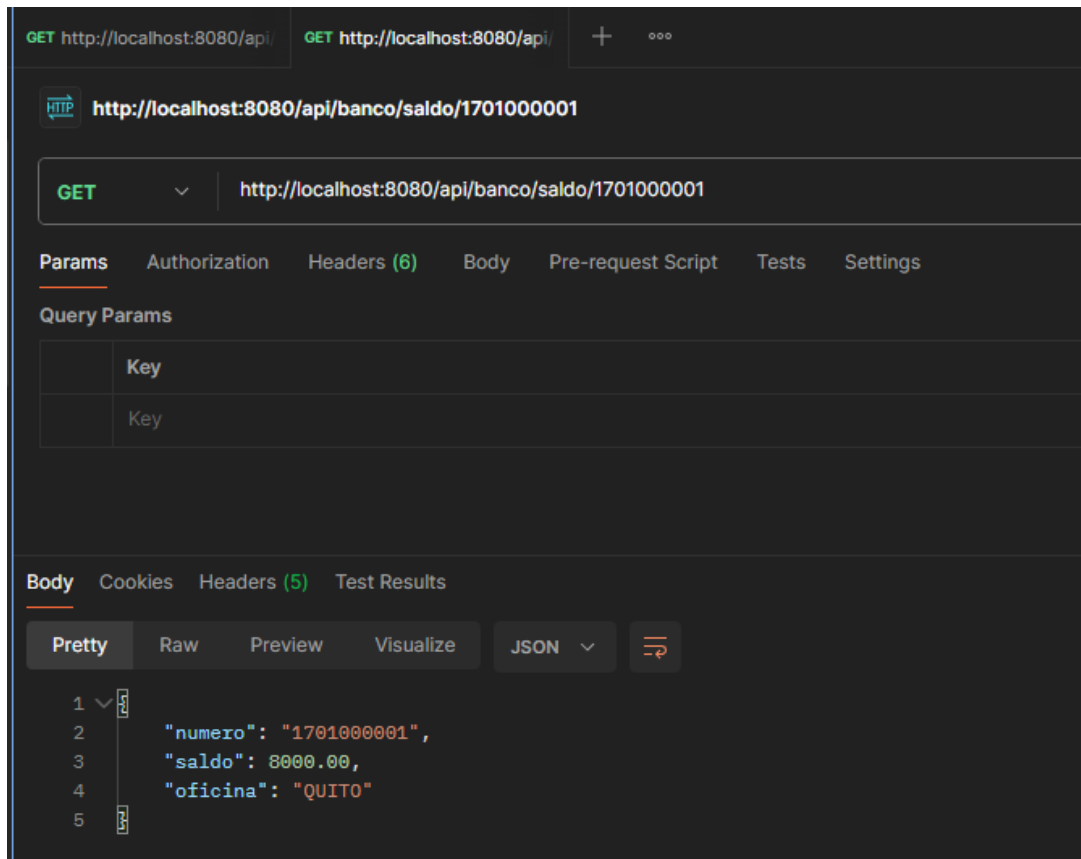


Figura 4: Consulta de saldo dirigida al nodo Quito.

7.4. Consulta distribuida de clientes

El endpoint de clientes devolvió seis registros: tres almacenados en Cuenca y tres almacenados en Quito.

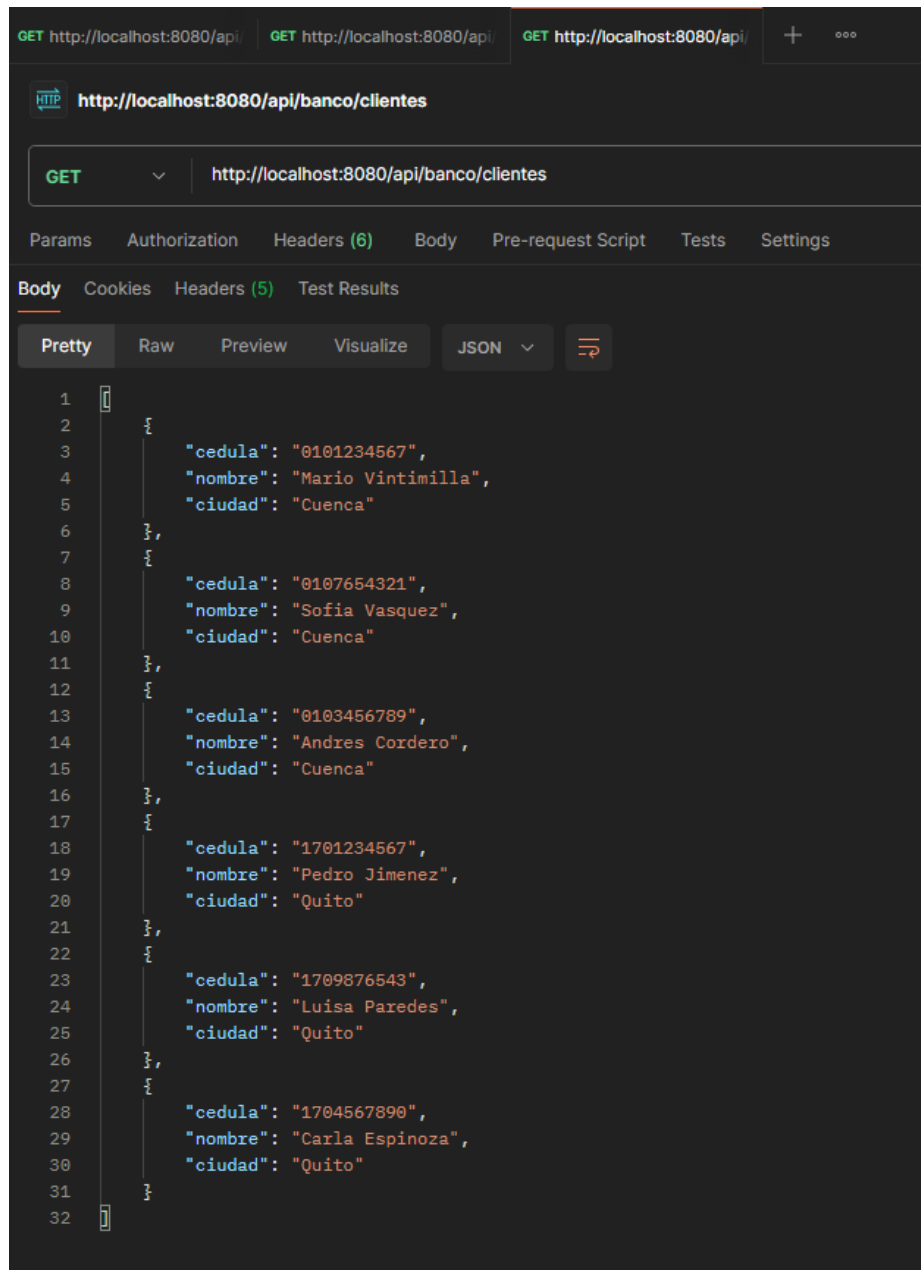


Figura 5: Clientes recuperados desde los dos nodos.

7.5. Prueba de autonomía local

El contenedor de Quito fue detenido de manera controlada. Durante la interrupción, la consulta de una cuenta de Cuenca continuó respondiendo con HTTP 200. La consulta dirigida a Quito devolvió HTTP 500, resultado esperado porque el prototipo no incorpora todavía un mecanismo de réplica o *fallback* para cuentas de una sede caída. Después de la prueba, Quito fue iniciado nuevamente y recuperó su operación normal.

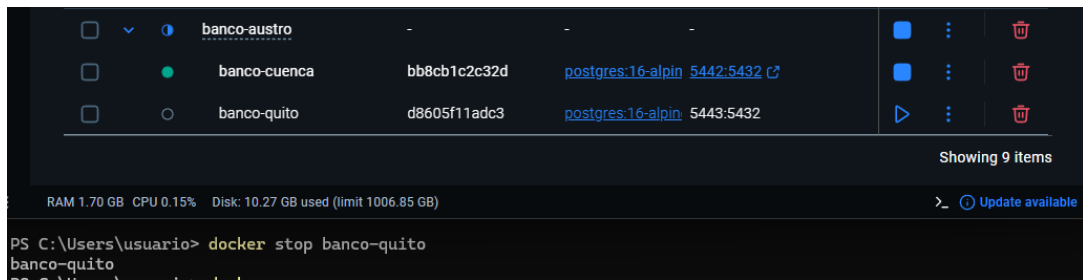


Figura 6: Nodo Quito detenido durante la prueba de fallos.

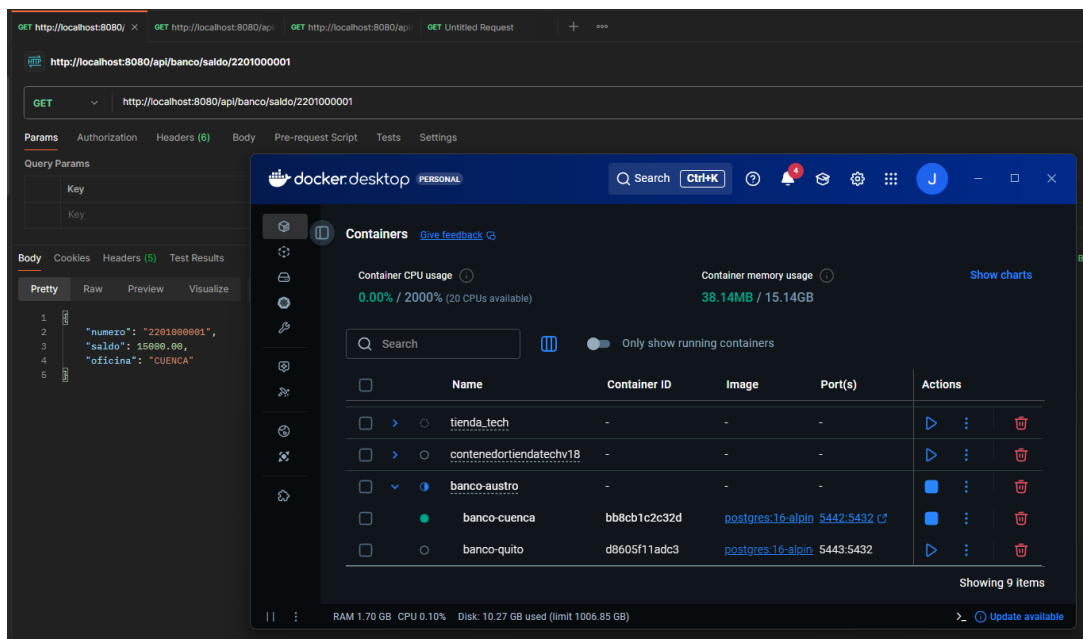


Figura 7: Cuenca continúa respondiendo mientras Quito está detenido.

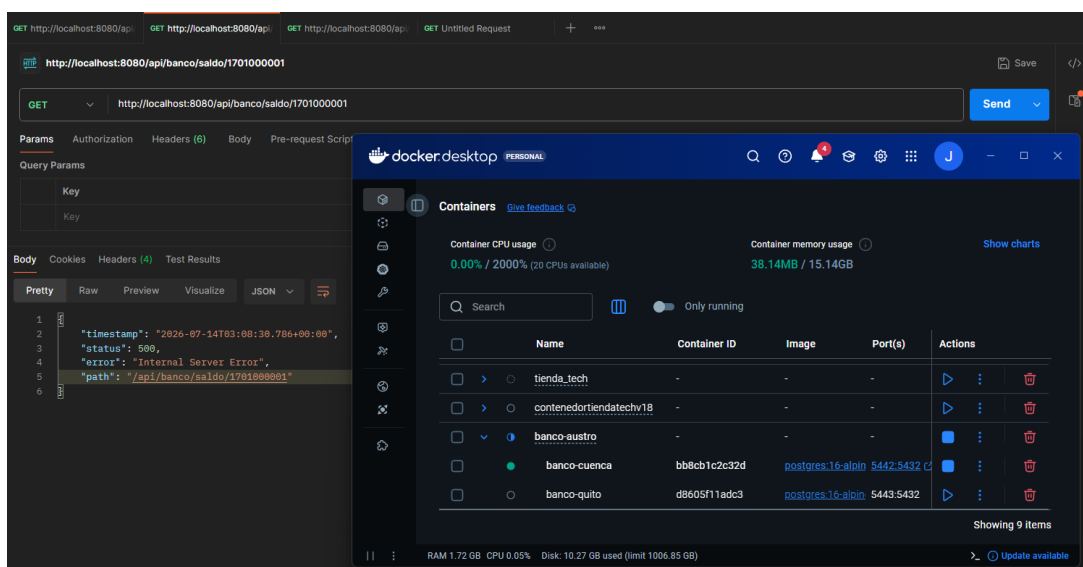


Figura 8: La consulta dirigida a Quito devuelve HTTP 500 mientras su nodo está detenido.

8. Análisis de resultados

Los resultados demuestran transparencia de ubicación porque el consumidor siempre consulta la misma API. La selección del nodo ocurre internamente y depende del prefijo de la cuenta. También se comprueba la disjunción de los fragmentos: cada sede acepta únicamente cuentas asociadas con su oficina.

La caída de Quito no impidió consultar cuentas de Cuenca, lo que evidencia autonomía local. Sin embargo, el endpoint de clientes depende actualmente de ambos nodos y falla si uno no está disponible. Esta limitación podría abordarse posteriormente mediante un patrón Circuit Breaker, una respuesta parcial controlada o replicación efectiva de los datos compartidos.

9. Conclusiones

1. Se desplegaron correctamente dos motores PostgreSQL independientes y persistentes.
2. La fragmentación horizontal permitió asignar las cuentas a una sede mediante restricciones verificables en la base de datos.
3. Spring Boot ofreció transparencia de ubicación al concentrar el acceso en una sola API REST.
4. Las consultas de Cuenca y Quito devolvieron los saldos esperados y la consulta conjunta recuperó seis clientes.
5. La prueba de fallos confirmó que un nodo disponible puede continuar atendiendo sus datos locales cuando el otro se encuentra detenido.

10. Recomendaciones

- Incorporar manejo global de excepciones para devolver mensajes controlados cuando un nodo no esté disponible.
- Implementar un Circuit Breaker para evitar esperas repetidas ante fallos de conexión.
- Evaluar replicación lógica para información que deba estar disponible en todas las sedes.
- Utilizar variables de entorno o secretos para administrar credenciales en ambientes no académicos.

11. Referencias

1. Guerrero Ulloa, G. *Práctica de Bases de Datos Distribuidas: Caso Banco del Austro*. Universidad Técnica Estatal de Quevedo, 2026.
2. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
3. Docker Inc. *Docker Compose Documentation*. <https://docs.docker.com/compose/>
4. VMware. *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>